PlusCal

P-syntax inside a TLA+ module · Parse translates, TLC checks · everything between two labels is one atomic step

PlusCal · TLA+ · TLC and .cfg · Watch out

Module shape Name.tla, with the algorithm in a comment

```
---- MODULE Name ----                          the file must be Name.tla
EXTENDS Naturals, Sequences, TLC               TLC gives assert and print
CONSTANTS N                                    value comes from Name.cfg
Op(x) == x + 1                                 cannot see the variables
(* --algorithm Name                            or --fair algorithm
variables x = 0, s \in {1, 2};                 \in: TLC tries each value
define Inv == x >= 0 end define;               sees the variables and pc
macro Inc(v) begin v := v + 1; end macro;
process p \in 1..N                             set of processes, self = id
  variables mine = 0;                          per-process variable
begin
  L1: while TRUE do ... end while;             label here, and every while
end process;
process q = "logger" begin L: skip; end process;
end algorithm; *)
\* BEGIN TRANSLATION                           generated: Init, Next, Spec
\* END TRANSLATION                             never edit this region
====
```

Statements end every one with ;

- `x := e;` assign; = compares, := assigns
- `f[1] := a || f[2] := b;` two writes, one step
- `if c then … else … end if;` elsif c then … too
- `while c do … end while;` always needs a label
- `either … or … end either;` TLC tries every arm
- `with v \in S do … end with;` each v; or v = e
- `await c;` step blocked until c (also `when`)
- `skip;` `goto L;` nothing / jump, ends the step
- `assert e;` TLC error if false (EXTENDS TLC)
- `print e;` to TLC's output
- `call P(a);` `return;` procedure; ends the step

Labels where a step begins

One label to the next is **one atomic step**; other processes run only at a label. Too few labels hide bugs, too many add states but stay sound.

Required: the first statement of the body, each process, each procedure; every `while`; after `goto`, `call`, `return`; a second write to a variable in a step (or use `||`); after an `if` / `either` containing a label.

Forbidden inside `with` and a macro. All are reported as `Missing label at …` with a line and column.

Properties liveness needs fairness

- `INVARIANTS` **invariant:** holds in every state
- `PROPERTIES` **temporal:** holds on every behaviour
- `[]P` always `<>P` eventually
- `P ~> Q` leads to: whenever P, Q then or later
- `[]<>P` infinitely often `<>[]P` eventually forever

- `fair process` weak (**WF**): a step enabled continuously is eventually taken.
- `fair+ process` strong (**SF**): enabled infinitely often, gaps allowed, is taken.
- `--fair algorithm` WF of all Next: one process only; else **fair process**.

No fairness: safety checks, liveness fails.

TLA+ inside PlusCal expressions, define, invariants

LOGIC	`/\ \/ ~ => <=> = #  IF c THEN a ELSE b`
NUMBERS	`+ - * \div % < <= > >= 1..N`
SETS	`{1, 2} \in \notin \cup \cap S \ T` `\subseteq SUBSET S Cardinality(S)` FiniteSets `{x \in S : P(x)}` filter `{e(x) : x \in S}` map
QUANTIFIERS	`\A x \in S : P  \E x \in S : P` `CHOOSE x \in S : P` deterministic pick
FUNCTIONS	`[i \in S |-> e] f[i] DOMAIN f [S -> T]` `f[i] := e` in PlusCal is `[f EXCEPT ![i] = e]`
RECORDS	`[a |-> 1, b |-> 2] r.a r.a := 3` `[a : S, b : T]` set of records

SEQUENCES	`<< >> <<1, 2>> Append(s, e) Seq(S)`
	`Head(s) Tail(s) Len(s) s \o t s[i]`
	`SubSeq(s, m, n)` strings are just values

Name.cfg TLC's input and output

```
CONSTANTS
  N = 3
  Procs = {p1, p2} \* model values
SPECIFICATION Spec
INVARIANTS TypeOK Safe
PROPERTIES Termination Live
CHECK_DEADLOCK FALSE \* if wanted
```

```
No error has been found.
states generated / distinct / depth
Invariant X is violated.
trace State 1 … State n: follow pc
Temporal properties were violated.
ends in Back to state k, or Stuttering
Deadlock reached.
nothing enabled and not every process Done
The operator X is not defined
a .cfg name with no definition in the module
```